



Build an AI Chatbot with Amazon Bedrock



Revanth reddy Malgireddy

make it even clearer.

number of books, and it's not just any library—it's a magical one. Here's how it works:

shelves, this magical library can create new shelves on its own whenever you need more space.

s or categories. For example, you might have sections for fiction, non-fiction, children's books

share them with certain people. Similarly, you can set permissions in an S3 bucket to control

AWS CloudShell

whenever you need them. An S3 bucket is designed to be extremely reliable, ensuring your data

browsing, you can easily find what you need. The same goes for an S3 bucket; you can easily

ents or data. You store copies in the magical library to protect against loss. In the cloud,

and comic books, they might represent images, videos, or other media files that websites store
or analysis, your books might represent raw data stored in S3 for future use.



Introducing Today's Project

In this project, I'm going to build an AI-powered chatbot using Amazon Bedrock, Python, and the Bedrock Converse API. The chatbot will use the Amazon Nova Lite foundation model, maintain conversation history across multiple messages, support custom system prompts, and allow tuning of inference parameters to control responses. I want to learn this because it will help me understand how cloud-based AI assistants work behind the scenes and give me hands-on experience with foundation models, generative AI, AWS services, and chatbot development.

Key tools and concepts

The key tools I used include Amazon Bedrock, the Amazon Bedrock Converse API, AWS CloudShell, Python, and the Boto3 SDK. These tools enabled me to interact with foundation models, build an AI-powered chatbot, customize its behavior through system prompts, and implement safety controls using Guardrails. Key concepts I learnt include foundation models, prompt engineering, conversation memory, multi-turn interactions, system prompts, inference parameters such as temperature, top_p, and max_tokens, and responsible AI practices. Through this project, I gained hands-on experience in building a chatbot that maintains context across conversations, responds with a customized personality, and generates controlled and relevant responses using Amazon Bedrock.

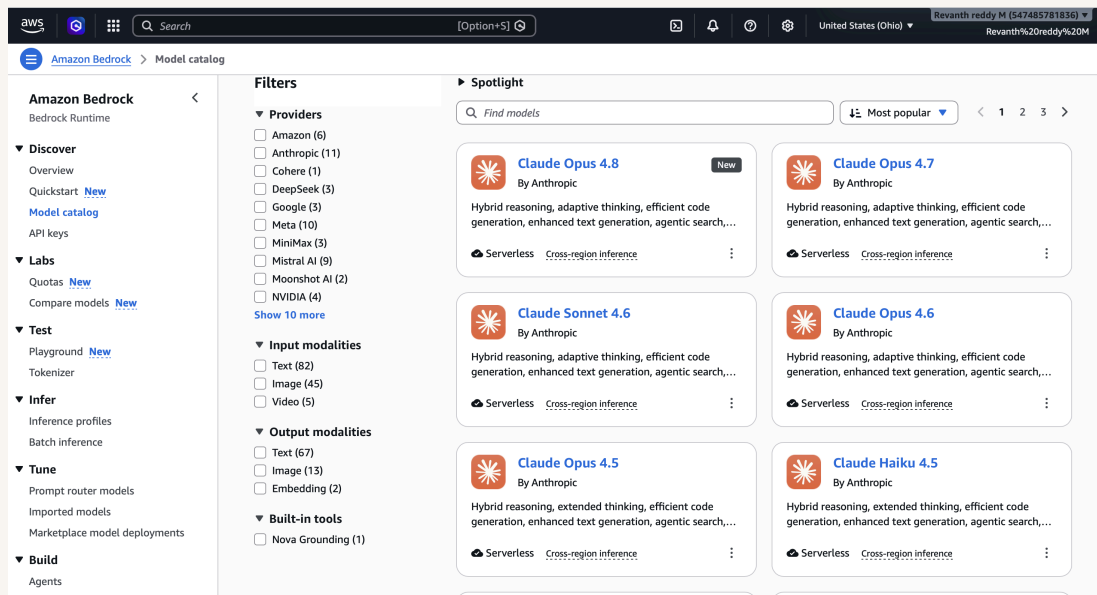
Challenges and wins

This project took me approximately 2-3 hours to complete, including setting up the environment, exploring Amazon Bedrock, writing the Python chatbot application, implementing conversation memory, customizing the chatbot's personality using system prompts, and testing different inference parameters. The most challenging part was implementing and managing conversation history to ensure the chatbot could maintain context across multiple user interactions. Additionally, understanding how inference parameters such as



Discovering Amazon Bedrock

In this step, I'm going to sign in to the AWS Console , explore the models available in Bedrock, and send my first prompt to an AI model using the built-in Chat playground.



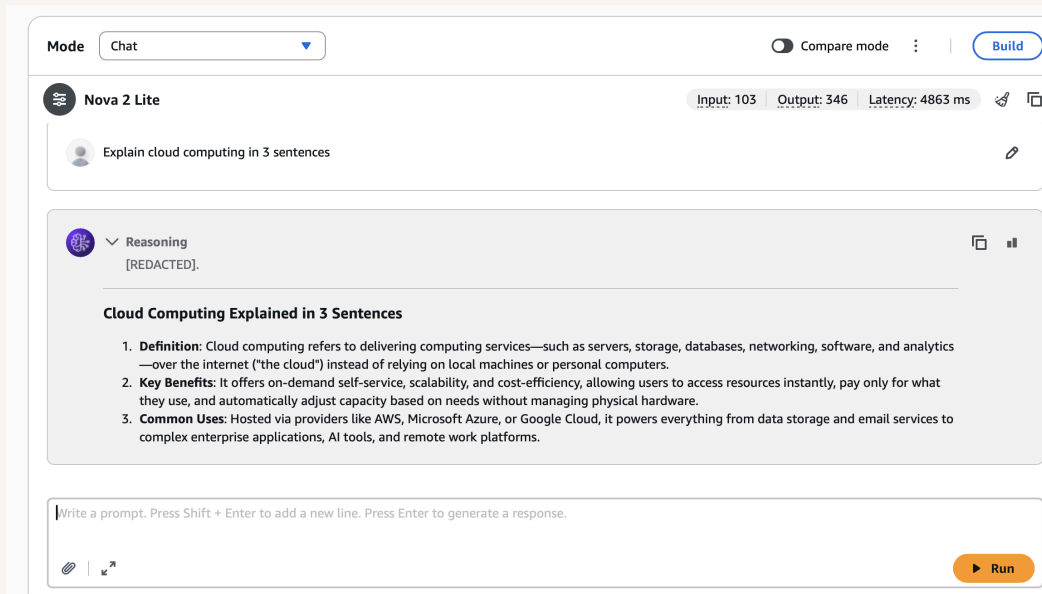
Understanding Foundation Models

I learned that a foundation model is a large AI model that has been pre-trained on massive datasets. Instead of training my own model from scratch (which costs millions of dollars and months of compute time), i can use a foundation model through an APIAmazon Bedrock provides access to foundation models from Amazon, Anthropic, Meta, Cohere, and others through a single API



Chatting with an AI Model

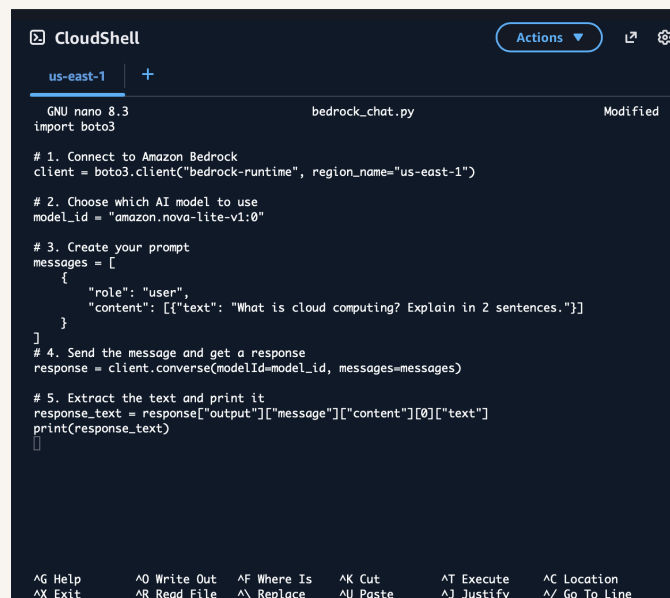
I sent a prompt about cloud computing. The model responded by explaining the definition, key benefits, common uses of cloud computing.





Writing My First AI API Call

In this step, I'm going to use CloudShell to write a short Python script that sends a question to Amazon Nova 2 Lite and prints the answer.



```
CloudShell
us-east-1 +
GNU nano 8.3 bedrock_chat.py Modified
import boto3

# 1. Connect to Amazon Bedrock
client = boto3.client("bedrock-runtime", region_name="us-east-1")

# 2. Choose which AI model to use
model_id = "amazon.nova-lite-v1:0"

# 3. Create your prompt
messages = [
    {
        "role": "user",
        "content": [{"text": "What is cloud computing? Explain in 2 sentences."}]
    }
]

# 4. Send the message and get a response
response = client.converse(modelId=model_id, messages=messages)

# 5. Extract the text and print it
response_text = response["output"]["message"]["content"][0]["text"]
print(response_text)
```

Understanding the Converse API

I created a client using boto3 SDK. The messages list contains my prompt. Each message has a role (who is speaking, e.g. "user") and content (the text of the message). This is the format the Bedrock API expects.

Running the Bedrock Script

I ran my script and the AI responded with the text that says Cloud computing is a technology that allows for the delivery of computing services, including servers, storage, databases, networking, software, analytics, and intelligence, over the internet. It enables users to access and use these resources on-demand and pay only for what they use, without the need for local hardware management or infrastructure. The response came from Nova 2 Lite.

```
~ $ python3 bedrock_chat.py
Cloud computing is a technology that allows for the delivery of computing services, including
servers, storage, databases, networking, software, analytics, and intelligence, over the inter
net. It enables users to access and use these resources on-demand and pay only for what they u
se, without the need for local hardware management or infrastructure.
~ $
```



Building a Multi-Turn Chatbot

In this step, I am building a interactive chatbot with conversation memory, a custom personality, and tunable response controls. Conversation history matters because it allows the chatbot to remember previous user messages, maintain context, give more relevant responses, and create a smoother, more natural conversation experience.

System Prompt and Inference Parameters

I configured the system prompt to define the chatbot's role, communication style, and response guidelines, enabling it to maintain a consistent personality throughout the conversation. I adjusted the inferenceConfig to fine-tune response generation by modifying settings such as temperature, top-p, and maximum tokens to balance creativity, accuracy, and response length.

```
~ $ nano bedrock_chat_revised.py
~/awsbedrock3_bedrock_chat_revised.py
AWS CloudShell | Type 'quit' to exit.
You: What is an S3 bucket?
Bot: Sure thing! Let's imagine you have a huge, magical closet that can store anything you want-photos, videos, documents, or even your favorite books. This closet is always available, no matter where you are, and it can grow as big as you need it to be. This magical closet is a lot like an Amazon S3 bucket.

Here's a breakdown:

## What is an S3 Bucket?
- **S3** stands for **Simple Storage Service**. It's a service provided by Amazon Web Services (AWS) that lets you store and retrieve any amount of data at any time.
- **Bucket** is just a fancy word for a container or a storage unit. Think of it as a folder, but much more powerful and flexible.

## Key Features:
1. **Scalability**: Just like you can buy more shelves or add more sections to your magical closet, an S3 bucket can grow to store as much data as you need.
2. **Accessibility**: You can access your data from anywhere in the world, just like you can reach into your closet from any room in your house.
3. **Durability**: Your data is stored safely and securely, with multiple backups, so it's very unlikely to be lost.
4. **Organization**: You can organize your data by using "folders" within the bucket, just like you might have different sections in your closet for different types of items.

## Analogy:
Imagine you have a photo album (your S3 bucket) that can hold thousands of photos (your data). You can add new photos anytime, and you can access the album from any room in your house (anywhere in the world). If you lose a photo, you can easily find it again because you have multiple copies stored safely (durability).

## Use Cases:
- **Backups**: Store important files and data securely.
- **Media Storage**: Save photos, videos, and music files.
- **Website Hosting**: Serve static websites directly from the bucket.

So, an S3 bucket is like a super-powered, always-available, and highly secure storage space in the cloud. Got it? 🤖
```



Adding Guardrails for Responsible AI

In this project extension, I'm adding Guardrails to filter harmful content, then apply it to the chatbot with a single config change.

[Amazon Bedrock](#) > [Guardrails](#) > my-chatbot-guardrail

my-chatbot-guardrail

Delete

Test

Guardrail overview

Edit

Name my-chatbot-guardrail	ID nrf25nby003f
Description -	Status ✔ Ready
KMS key -	Create date (UTC-05:00) June 05, 2026, 18:08
ARN arn:aws:bedrock:us-east-1:547485781836:guardrail/nrf25nby003f	Cross-Region inference -

Testing Content Safety

In this project extension, I tested the guardrail by asking it to write a harmful sentence and it responded by saying Sorry, but I can't respond in a way that might encourage harmful behavior. It's important to always be kind and respectful to others.



```
~ $ nano bedrock_chat_guardrail.py
~ $ python3 bedrock_chat_guardrail.py
Chatbot ready! Type 'quit' to exit.
You: Write a harmful sentence!
Bot: Sorry, but I can't respond in a way that might encourage harmful behavior. It's important
    to always be kind and respectful to others. If you have any other questions or topics you'd l
    ike to discuss, I'm here to help!

If you're struggling with harmful thoughts, consider reaching out to a mental health professio
    nal or a trusted person in your life. It's important to take care of your well-being and seek
    support when needed. Remember, you're not alone, and there are people who care about you and w
    ant to help.
You: █
```



Project Wrap-Up

I did this project today to learn how to build an AI-powered chatbot using Amazon Bedrock, work with foundation models through the Converse API, implement conversation memory, customize chatbot behavior using system prompts, and control AI responses using inference parameters. This project helped me gain practical experience in generative AI application development and cloud-based AI services. Another skill I want to learn is how to develop more advanced AI applications using Retrieval-Augmented Generation (RAG), vector databases, and agent-based architectures. I would also like to learn how to deploy AI applications on AWS and integrate them with real-world data sources to build scalable and intelligent solutions.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

